



## ***ClosingScript***

This project may appeal to programmers who dislike CoffeeScript's reliance on significant indentation but otherwise appreciate the language for its simplicity and expressiveness.

***ClosingScript*** addresses this issue by adding closing keywords to CoffeeScript syntax to terminate statements, making indentation no longer significant.

***ClosingScript*** is a text preprocessor that regenerates significant indentation using closing keywords. Its output can be piped directly into the official CoffeeScript compiler to make a one-step process: conversion → compilation → JavaScript.

A closing keyword consists of the slash followed by the name of its matching statement. This is borrowed from HTML closing tags.

List of closing keywords:

***/if, /unless, /while, /until, /for, /switch, /loop, /try, /class, /end*** (function)

Advantages:

- Whitespace-insensitive syntax.
- Robust code structure.
- Clear visual termination of statements.
- Safe copy-pasting.
- Safe auto-formatting.

Disadvantages:

- Slightly more verbose syntax.

## Syntax

The syntax is identical to CoffeeScript syntax except for the addition of a closing keyword for statements. The list below is not exhaustive and is provided to give a general idea.

```
functionName = (parameters) -> | =>      # no trailing comment allowed
  code
/end [optional trailing code]            # example of with trailing code: /end, timeout
```

```
do ->
  code
/end
```

```
if condition
  code
[else if condition
  code]
[else
  code]
/if
```

```
while condition [guard clauses]
  code
/while
```

```
for iterable [guard clauses]
  code
/for
```

```
loop
  code
/loop
```

```
switch [variable]
when condition
  code
[when condition then code]
[else
  code]
/switch
```

```
class name
  code
/class
```

```
try
  code
[catch ...
  code]
[finally ...
  code]
/try
```

## One-liners

**ClosingScript** always opens a block when it detects a **if**, **unless**, **while**, **until**, **for** or **loop** statement in the standard *prefix* version.

To force a statement as a one-liner, add a colon to the statement keyword. This also makes a line immediately recognizable as a one-liner when reading a source without looking further.

Ex:

```
if: condition then code [else code]
```

Or use a *postfix* version if applicable.

## Assignment from multi-line constructs

The first line of a multi-line construct of a statement, a heredoc, and a block comment must stand alone on its line to be correctly processed.

Assigning the return value of a multi-line construct requires placing the target variable on the preceding line.

```
variable =  
  """ heredoc on  
    multiple lines  
  """
```

```
variable =  
  switch variable  
  ...  
  /switch
```

## Usage

The preprocessor outputs to *stdout*. To get JavaScript, the official CoffeeScript compiler must be installed.

Any error message with a line number from the preprocessor or the compiler will match the same line in a ***ClosingScript*** source.

Convert and compile to JavaScript in one step:

```
closing script.coffee | coffee -s -c > script.js
```

Get a converted copy of a source:

```
closing script.coffee > script.converted.coffee
```

Get a formatted copy of a source (only reindented consistently):

```
closing -f script.coffee > script.formatted.coffee
```

## Usage with the Geany code editor

In Linux, this command can be added to the Build menu of the Geany code editor. Its converts and compiles in one step, with errors falling back at guilty lines in the editor window, as well as error messages from the preprocessor and the compiler are sent to the ‘Compiler’ message window. This conveniently makes the whole process as a single compiling step to the user.

```
set -o pipefail; closing "%f" | coffee -s -c > "%e".js    2> >(sed 's\[stdin\]/%f/' >&2)
```